



Django MongoDB Backend

Building Django apps on MongoDB with
Django MongoDB Backend

```
pip install django-mongodb-backend
```

django-mongodb-backend 6.0.3 · Django 6.0.5 · 🤖 assisted



Agenda

- Install library & start project
- Start MongoDB local or Atlas
- Meet Django MongoDB Backend
- Why MongoDB for a Django app?
- Modeling: embedded docs, arrays, ObjectIds
- Querying & aggregation
- Indexes & unique constraints
- What's new in 6.0: encryption, geo, polymorphism
- Limitations & resources
- Demo: `polls` app



Install library

```
# Install the latest version  
pip install django-mongodb-backend
```

```
# Match the version to your Django series  
pip install "django-mongodb-backend==6.0.*" # Django 6.0  
pip install "django-mongodb-backend==5.2.*" # Django 5.2
```



Start project

```
# Use the Django MongoDB Backend project template.  
django-admin startproject -v 3 demo \  
  --template https://github.com/mongodb-labs/django-mongodb-p
```

 Copy command

The project template configures `DATABASES` (`HOST` hardcoded to `mongodb://localhost:27017/`), `DEFAULT_AUTO_FIELD` (`ObjectIdAutoField` for every model), `INSTALLED_APPS` (custom `AppConfig` subclasses so contrib apps use ObjectId PKs), `MIGRATION_MODULES` (ObjectId-friendly copies of auth, admin & contenttypes migrations), and `DATABASE_ROUTERS` (`MongoRouter` to keep embedded models off `migrate`).



Start project 🤔

```
ast login: Sun May 10 14:21:47 on ttys001
[oh-my-zsh] Random theme 'gallois' loaded
[~]$ django-admin startproject -v 3 demo \
--template https://github.com/mongodb-labs/django-mongodb-project/archive/refs/heads/6.0.x.zip
Rendering project template files with extensions: .py
Rendering project template files with filenames: ???
Downloading https://github.com/mongodb-labs/django-mongodb-project/archive/refs/heads/6.0.x.zip
Extracting /var/folders/tp/d_4lszh1793941bn1mrrs4m0000gn/T/django_project_template_my0_vd0r_download/django-mongodb-project-6.0.x.zip
Creating /Users/alex.clark/demo/README.md
Creating /Users/alex.clark/demo/manage.py
Creating /Users/alex.clark/demo/demo/__init__.py
Creating /Users/alex.clark/demo/demo/urls.py
Creating /Users/alex.clark/demo/demo/asgi.py
Creating /Users/alex.clark/demo/demo/apps.py
Creating /Users/alex.clark/demo/demo/wsgi.py
Creating /Users/alex.clark/demo/demo/settings.py
Creating /Users/alex.clark/demo/mongo_migrations/__init__.py
Creating /Users/alex.clark/demo/mongo_migrations/auth/__init__.py
Creating /Users/alex.clark/demo/mongo_migrations/auth/0001_initial.py
Creating /Users/alex.clark/demo/mongo_migrations/admin/__init__.py
Creating /Users/alex.clark/demo/mongo_migrations/admin/0001_initial.py
Creating /Users/alex.clark/demo/mongo_migrations/contenttypes/__init__.py
Creating /Users/alex.clark/demo/mongo_migrations/contenttypes/0001_initial.py
Cleaning up temporary files.
[640ms] [~]$
```



Rendering project template files

Django's `TemplateCommand` renders a file if either is true:

- Its extension is in `--extension` (default: `py`)
- Its filename is in `extra_files` — populated via `--name`

Use `--name` for files without a useful extension:

```
django-admin startproject -v 3 demo \  
  --name Dockerfile --name Procfile \  
  --template https://github.com/mongodb-labs/django-mongodb-p
```



Start MongoDB local or Atlas

Use `mongodb-runner` local

```
# Install once
npm install -g mongodb-runner

# Standalone server on the default port
mongodb-runner start

# Or a replica set (required for transactions)
mongodb-runner start -t replset

# List what's running, then stop everything
mongodb-runner ls
mongodb-runner stop --all

# Run a command with MONGODB_URI pre-set
mongodb-runner exec -t standalone -- sh -c 'python manage.py'
```

Requires Node.js ≥ 20.19. Downloads the MongoDB binary on first run, caches it after. Source: [mongodb-js/devtools-shared](https://github.com/mongodb-js/devtools-shared).



Start MongoDB local or Atlas

Inspect with `mongosh` — the official MongoDB shell, a JS REPL for ad-hoc queries.

```
# Install once (or `brew install mongosh`)
npm install -g mongosh

# Connect — accepts the same URI as the Django backend
mongosh "$MONGODB_URI"

# One-shot commands with --eval
mongosh "$MONGODB_URI" --quiet --eval 'db.movie.find({ year:
mongosh "$MONGODB_URI" --quiet --eval 'db.getSiblingDB("demo'

# Or pipe through mongodb-runner so MONGODB_URI is set for you
mongodb-runner exec -t standalone -- sh -c 'mongosh $MONGODB_
```

Replaces the legacy `mongo` shell. Also available via Homebrew and the MongoDB downloads page.



Configure **MONGODB_URI**

```
# settings.py
import os

DATABASES = {
    "default": {
        "ENGINE": "django_mongodb_backend",
        "HOST": os.environ.get("MONGODB_URI", "mongodb://localhost:27017"),
        "NAME": "demo",
    },
}

# Use ObjectId as the default primary key
DEFAULT_AUTO_FIELD = "django_mongodb_backend.fields.ObjectIdAutoField"

INSTALLED_APPS = [
    # ...
    "django_mongodb_backend",
]
```

HOST accepts any MongoDB connection string — including Atlas SRV URIs and TLS options. Read it from the environment so secrets stay out of the repo.



justfile

```
default:
    @just --list

_uri:
    ...

startproject:
    django-admin startproject demo -v 3 --template templates/project_template

runserver:
    cd demo && MONGODB_URI="$(just _uri)" python manage.py runserver

migrate:
    cd demo && MONGODB_URI="$(just _uri)" python manage.py migrate

dropdb:
    mongosh "$(just _uri)" --quiet --eval 'db.getSiblingDB("demo").dropDatabase()'
```

`$(just _uri)` calls a private recipe that finds (or starts) a `mongodb-runner` instance and prints its URI.



Meet Django MongoDB Backend



- Built on PyMongo
- Tracks Django releases: 5.2 6.0
- A first-class Django database backend
 - Not an ODM bolt-on or sidecar
 - Listed in Django's third-party database backends
 - Translates the ORM to MQL
 - `str(qs.query)` prints MQL



ORM → MQL

As of 6.0, `str(qs.query)` prints the aggregation pipeline.

```
>>> qs = (Question.objects
...       .filter(question_text__icontains="mongo")
...       .order_by("-pub_date")[:5])
>>> print(qs.query)
[
  {'$match': {'$expr': {'$regexMatch': {
    'input': '$question_text', 'regex': 'mongo', 'options':
  {'$sort': {'pub_date': -1}},
  {'$limit': 5}
]
```

Each node in Django's query tree gets an `as_mql()` method (mirroring `as_sql()`); the compiler walks the tree to build the pipeline. Use `qs.explain()` for the executed plan.



ORM → MQL

Content types & permissions via the Django ORM.

```
from django.contrib.auth.models import Permission, User
from django.contrib.contenttypes.models import ContentType

# Every installed model has a ContentType row
qs = ContentType.objects.filter(app_label="auth")
str(qs.query)
# [{'$match': {'app_label': {'$eq': 'auth'}}}]

# ContentType for a specific model class
ct = ContentType.objects.get_for_model(User)

# Auth permissions joined to their content type – one query
perms = Permission.objects.select_related("content_type").filter(
    content_type__app_label="auth"
)
```



Copy



ObjectId primary keys

`DEFAULT_AUTO_FIELD` configures the field used for auto-generated PKs.

BigAutoField

- 64-bit integer counter
- Database hands out the next value
- Awkward under sharding

UUIDField

- 128-bit random value
- Generated client-side
- No time order

ObjectIdAutoField

- 12-byte BSON `ObjectId`
- Timestamp-prefixed — sorts ~chronologically
- Generated client-side, safe under sharding

ObjectId = 4-byte Unix timestamp · 5-byte random · 3-byte counter (12 bytes total).



Global vs per-app

Set it once in `settings.py` — or per app in `AppConfig`.

`settings.py`

```
DEFAULT_AUTO_FIELD = \
    "django_mongodb_backend.fields.ObjectIDField"
```

- Project-wide default
- Applies to every app that doesn't override

`apps.py`

```
from django.apps import AppConfig

class BlogConfig(AppConfig):
    default_auto_field = "django_mongodb_backend.fields.ObjectIDField"
    name = "blog"
```

- Per-app override
- Wins over the global setting

Model `pk` > App `default_auto_field` > `DEFAULT_AUTO_FIELD`



Migrations & contrib apps

What the project template adds to `settings.py`.

INSTALLED_APPS

```
INSTALLED_APPS = [  
    "demo.apps.MongoAdminConfig",  
    "demo.apps.MongoAuthConfig",  
    "demo.apps.MongoContentTypesConfig",  
    "django.contrib.sessions",  
    "django.contrib.messages",  
    "django.contrib.staticfiles",  
    "django_mongodb_backend",  
]
```

- contrib AppConfigs swap in `ObjectIdAutoField`
- `django_mongodb_backend` ships checks & commands

MIGRATION_MODULES

```
MIGRATION_MODULES = {  
    "admin": "mongo_migrations.admin",  
    "auth": "mongo_migrations.auth",  
    "contenttypes": "mongo_migrations.contentt  
}  
  
DATABASE_ROUTERS = [  
    "django_mongodb_backend.routers.MongoRoute  
]
```

- Reconfigures contrib migrations to use ObjectId-friendly copies
- `MongoRouter` keeps embedded models off `migrate`

All of this is what `startproject --template ...` writes for you — but you can manually add it to an existing project or a new project created without the template.



The contrib AppConfigs

The `INSTALLED_APPS` entries above are defined in your project's `apps.py`.

```
from django.contrib.admin.apps import AdminConfig
from django.contrib.auth.apps import AuthConfig
from django.contrib.contenttypes.apps import ContentTypesConfig

class MongoAdminConfig(AdminConfig):
    default_auto_field = "django_mongodb_backend.fields.ObjectIdAutoField"

class MongoAuthConfig(AuthConfig):
    default_auto_field = "django_mongodb_backend.fields.ObjectIdAutoField"

class MongoContentTypesConfig(ContentTypesConfig):
    default_auto_field = "django_mongodb_backend.fields.ObjectIdAutoField"
```

Three one-line subclasses — exactly the per-app override pattern from the previous slide.



What the project template does

Action

- Subclass `AuthConfig`,
`AdminConfig`,
`ContentTypesConfig`
- Reconfigure
`MIGRATION_MODULES` to use
`mongo_migrations.*`
- Add `MongoRouter` to
`DATABASE_ROUTERS`

Reason

- Contrib AppConfigs default to `BigAutoField`; subclassing overrides to `ObjectIdAutoField`
- Bundled migrations assume integer primary keys; reconfigured migrations use `ObjectIdAutoField`
- Embedded models have no backing collection and must be excluded from routing

The project template includes all three of these changes.



Why MongoDB?

Document model

- Schema *fits the object*, not the other way around
- Nested documents & arrays — no join tables
- Flexible evolution without painful migrations

Operations

- Horizontal scale via sharding
- Replica sets & transactions
- Atlas: managed, with search & vector search



Django on MongoDB?

- Keep what you love: ORM, admin, migrations, forms, auth
- Gain document-native modeling:
`EmbeddedModelField`, `ArrayField`,
`ObjectIdField`
- Drop into MQL/aggregation when you need it
(`raw_aggregate`)
- Officially maintained by MongoDB

Repo: github.com/mongodb/django-mongodb-backend



Your first model

```
from django.db import models

class Movie(models.Model):
    title = models.CharField(max_length=200)
    year = models.IntegerField()
    genres = models.JSONField(default=list)
    runtime = models.IntegerField(null=True)

    class Meta:
        indexes = [models.Index(fields=["year", "title"])]
```

A standard Django model — no new API to learn. The backend stores each instance as a MongoDB document, and `id` is automatically a BSON `ObjectId` rather than an integer.



Embedded documents

```
from django_mongodb_backend.fields import EmbeddedModelField
from django_mongodb_backend.models import EmbeddedModel

class Address(EmbeddedModel):
    street = models.CharField(max_length=200)
    city = models.CharField(max_length=100)
    zipcode = models.CharField(max_length=20)

class Customer(models.Model):
    name = models.CharField(max_length=120)
    address = EmbeddedModelField(Address)
```

`Address` is stored inside the `Customer` document — no separate collection, no join. `customer.address.city` is a plain Python attribute lookup that maps directly to a MongoDB field path.



Arrays of embedded docs

```
from django_mongodb_backend.fields import (
    ArrayField,
    EmbeddedModelArrayField,
)

class LineItem(EmbeddedModel):
    sku = models.CharField(max_length=40)
    qty = models.IntegerField()
    price = models.DecimalField(max_digits=8, decimal_places=2)

class Order(models.Model):
    customer = models.ForeignKey(Customer, on_delete=models.CASCADE)
    items = EmbeddedModelArrayField(LineItem)
    tags = ArrayField(models.CharField(max_length=30), default=list)
```

Loading an `Order` fetches its embedded `LineItem` array and scalar `tags` in a single document read — no joins, no extra queries.



Querying

```
# Top-level fields
Movie.objects.filter(year__gte=2020).order_by("-year")[:10]

# Reach into embedded docs with double-underscore
Customer.objects.filter(address__city="Berlin")

# Array membership
Order.objects.filter(tags__contains=["priority"])

# See the MQL the ORM produced
print(Movie.objects.filter(year=2024).query)
```



When you need MQL:

raw_aggregate

```
pipeline = [  
    {"$match": {"year": {"$gte": 2020}}},  
    {"$group": {"_id": "$year", "count": {"$sum": 1}}},  
    {"$sort": {"_id": -1}},  
]  
  
qs = Movie.objects.raw_aggregate(pipeline) # returns a QuerySet  
for row in qs:                             # row is a Movie object  
    print(row.year, row.count)             # _id + annotated field
```

Full power of the aggregation framework — `$lookup`, `$search`, `$vectorSearch`, ...



Indexes — including embedded fields

```
from django_mongodb_backend.indexes import EmbeddedFieldIndex

class Customer(models.Model):
    name = models.CharField(max_length=120)
    address = EmbeddedModelField(Address)

    class Meta:
        indexes = [
            models.Index(fields=["name"]),
            EmbeddedFieldIndex("address.city"),
        ]
```

New in 6.0.2: index/uniqueness on subfields of embedded & embedded-array fields.



Unique constraints

```
from django.db.models import UniqueConstraint
from django_mongodb_backend.constraints import EmbeddedFieldUniqueConstra

class Customer(models.Model):
    email = models.EmailField(null=True, blank=True)
    address = EmbeddedModelField(Address)

    class Meta:
        constraints = [
            # Multiple NULLs allowed (6.0.2+)
            UniqueConstraint(
                fields=["email"],
                name="uniq_email",
                nulls_distinct=True,
            ),
            EmbeddedFieldUniqueConstraint("address.zipcode"),
        ]
```

Indexes do the work; the constraint is a thin wrapper over a unique index.



Polymorphic embedded models

6.0.3

```
from django_mongodb_backend.fields import (
    PolymorphicEmbeddedModelField,
    PolymorphicEmbeddedModelArrayField,
)

class CardPayment(EmbeddedModel):
    last4 = models.CharField(max_length=4)

class WirePayment(EmbeddedModel):
    iban = models.CharField(max_length=34)

class Invoice(models.Model):
    payment = PolymorphicEmbeddedModelField([CardPayment, WirePayment])
    history = PolymorphicEmbeddedModelArrayField([CardPayment, WirePayment])
```

Indexes & unique constraints on polymorphic subfields are also supported.



Queryable Encryption 6.0.1

- Encrypt fields client-side; query them *without* decrypting server-side
- Declared on the model, enforced by the backend
- Migrations refuse to run against a non-encrypted database

```
from django_mongodb_backend.encryption import EncryptedCharField

class Patient(models.Model):
    name = models.CharField(max_length=120)
    ssn = EncryptedCharField(max_length=11, queries={"queryTy
```



Spatial lookups

6.0.1

```
from django.contrib.gis.geos import Point
from django.contrib.gis.measure import D

# 2dsphere index under the hood
Cafe.objects.filter(
    location__dwithin=(Point(13.405, 52.52), D(km=2)),
)
```

GeoDjango-style queries, MongoDB-native execution.



The Django admin... still works!

- Register models exactly as you always have
- Embedded fields render as nested forms
- `ObjectId` primary keys are matched in URLs out of the box 6.0.3

```
# urls.py
urlpatterns = [
    path("orders/<objectid:pk>/", OrderDetail.as_view()),
]
```



Limitations

- No SQL — some third-party apps that emit raw SQL won't work
- Cross-collection joins exist, but you'll lean on embedding
- `contrib.gis` support is partial (lookups, not full GeoDjango)
- Schema migrations are lighter — MongoDB doesn't enforce a fixed shape

Check the docs' "Known issues" page before porting a large app.



Demo: `polls` app

The Django tutorial's polls app, ported to `django-mongodb-backend`.

- `EmbeddedModel` + `EmbeddedModelArrayField` — `Choice` embedded in `Question.choices`
- `ArrayField` — `Question.tags`
- `ObjectIdAutoField` — set on `PollsConfig`
- `MongoManager.raw_aggregate` — analytics view
- `SearchIndex` + `SearchText` — Atlas Search, with `__icontains` fallback

polls/models.py

```
from django.db import models
from django_mongodb_backend.fields import ArrayField, EmbeddedModelArrayField
from django_mongodb_backend.indexes import SearchIndex
from django_mongodb_backend.managers import MongoManager
from django_mongodb_backend.models import EmbeddedModel

class Choice(EmbeddedModel):
    choice_text = models.CharField(max_length=200)
    votes = models.IntegerField(default=0)

class Question(models.Model):
    question_text = models.CharField(max_length=200)
    pub_date = models.DateTimeField("date published")
    choices = EmbeddedModelArrayField(Choice, blank=True, default=list)
    tags = ArrayField(models.CharField(max_length=50), blank=True, default=list)

    objects = MongoManager()

class Meta:
    indexes = [
        SearchIndex(
            name="polls_question_search",
            field_mappings={
                "question_text": {"type": "string"},
                "tags": {"type": "string"},
            },
        ),
    ]
```

One collection, one document per question — choices and tags live inside.



Voting on an embedded **Choice**

```
def vote(request, pk):
    question = get_object_or_404(Question, pk=pk)
    try:
        index = int(request.POST["choice"])
        choice = question.choices[index]
    except (KeyError, ValueError, IndexError):
        return HttpResponseRedirect(reverse("polls:detail", args=(question.pk,)))
    choice.votes = (choice.votes or 0) + 1
    question.choices[index] = choice
    question.save()
    return HttpResponseRedirect(reverse("polls:results", args=(question.pk,)))
```

Mutate the embedded **Choice** in place, reassign it, save the parent.



raw_aggregate in a view

```
class AnalyticsView(TemplateView):
    template_name = "polls/analytics.html"

    def get_context_data(self, **kwargs):
        ctx = super().get_context_data(**kwargs)
        pipeline = [
            {"$project": {
                "question_text": 1, "pub_date": 1, "choices": 1, "tags": 1,
                "year": {"$year": "$pub_date"},
                "votes": {"$sum": "$choices.votes"},
            }},
            {"$sort": {"pub_date": -1}},
        ]
        questions = list(Question.objects.raw_aggregate(pipeline))
        per_year = {}
        for q in questions:
            per_year.setdefault(q.year, 0)
            per_year[q.year] += q.votes
        ctx["questions"] = questions
        ctx["per_year"] = sorted(per_year.items())
        return ctx
```

`$sum: "$choices.votes"` sums across the embedded array — no extra queries.



Atlas Search with a fallback

```
class SearchView(TemplateView):
    template_name = "polls/search.html"

    def get_context_data(self, **kwargs):
        ctx = super().get_context_data(**kwargs)
        q = (self.request.GET.get("q") or "").strip()
        ctx["q"], ctx["results"], ctx["mode"] = q, [], "none"
        if not q:
            return ctx
        try:
            from django_mongodb_backend.expressions.search import SearchText
            ctx["results"] = list(
                Question.objects.annotate(_search=SearchText("polls_question__text", q))
            )
            ctx["mode"] = "atlas"
        except Exception:
            ctx["results"] = list(Question.objects.filter(question_text__icontains=q))
            ctx["mode"] = "fallback"
        return ctx
```

Atlas: `$search`. Local replica set: `__icontains` + a banner.



Run the polls demo

```
# In your django-mongodb-backend project
pip install -e path/to/polls

# settings.py: INSTALLED_APPS += ["polls"]
# urls.py:      path("polls/", include("polls.urls"))

python manage.py makemigrations polls
python manage.py migrate
python manage.py seed_polls
python manage.py create_polls_search_index # Atlas only

python manage.py runserver # then visit /polls/
```



Resources

- Docs — django-mongodb-backend.readthedocs.io
- Source — github.com/mongodb/django-mongodb-backend
- Django MongoDB Project template — github.com/mongodb-labs/django-mongodb-project
- Atlas free tier — mongodb.com/atlas
- `mongodb-runner` — github.com/mongodb-js/devtools-shared
- `polls` app — github.com/aclark4life/polls
- `django-mongodb-extensions` — pypi.org/project/django-mongodb-extensions
- These slides — github.com/aclark4life/django-mongodb-slides





Thanks 🙌

Questions?

```
{ name      : "Jeffrey 'Alex' Clark",  
  title     : "Senior Software Engineer",  
  group     : "Database Experience Team",  
  email     : "alex.clark@mongodb.com" }
```

